



NumPy

Introduction of NumPy.

NumPy :- Numerical Python

Introduction :-

NumPy stands for Numerical Python. It is an open-source Python library used for numerical and scientific computing. NumPy provides a powerful multidimensional array object called `ndarray`, along with a large collection of mathematical functions to efficiently perform operations on arrays.

NumPy is one of the most important libraries in Python because many other libraries such as Pandas, SciPy, Matplotlib, Scikit-learn, and TensorFlow are built on top of it.

✓ **History of NumPy :-**

NumPy has a rich history that dates back to the early development of scientific computing in Python.

Early Development

Before NumPy existed, Python had limited support for numerical and scientific computations. To address this issue, two separate libraries were developed:

Numeric (released in 1995): The first major Python library for numerical computing.

Numarray (released in 2001): Designed to handle very large arrays more efficiently than Numeric.

Creation of NumPy

In 2005, Travis Oliphant, an American software developer and scientist, created NumPy by combining the best features of Numeric and Numarray into a single, unified library.

His goal was to provide:

- A faster numerical computing library
- Better memory efficiency
- A single standard array package for Python
- Compatibility with scientific applications

Double-click (or enter) to edit

Double-click (or enter) to edit

✓ Why NumPy was Developed?

Need for NumPy

Python is a simple and powerful programming language, but its built-in data structures, such as lists, are not designed for high-performance numerical computing. As the amount of data increases, operations on Python lists become slower and consume more memory.

NumPy was developed to overcome these limitations and provide a faster, more efficient way to perform numerical and scientific computations.

Problems with Python Lists

Python lists have several limitations when working with numerical data:

- Slower execution for mathematical operations.
- Higher memory consumption.

Why NumPy is Better

- NumPy solves these problems by providing a powerful multidimensional array object called **ndarray**.
- These arrays are stored in continuous memory locations, making computations much faster and more memory efficient than Python lists.

Double-click (or enter) to edit

Applications of NumPy :-

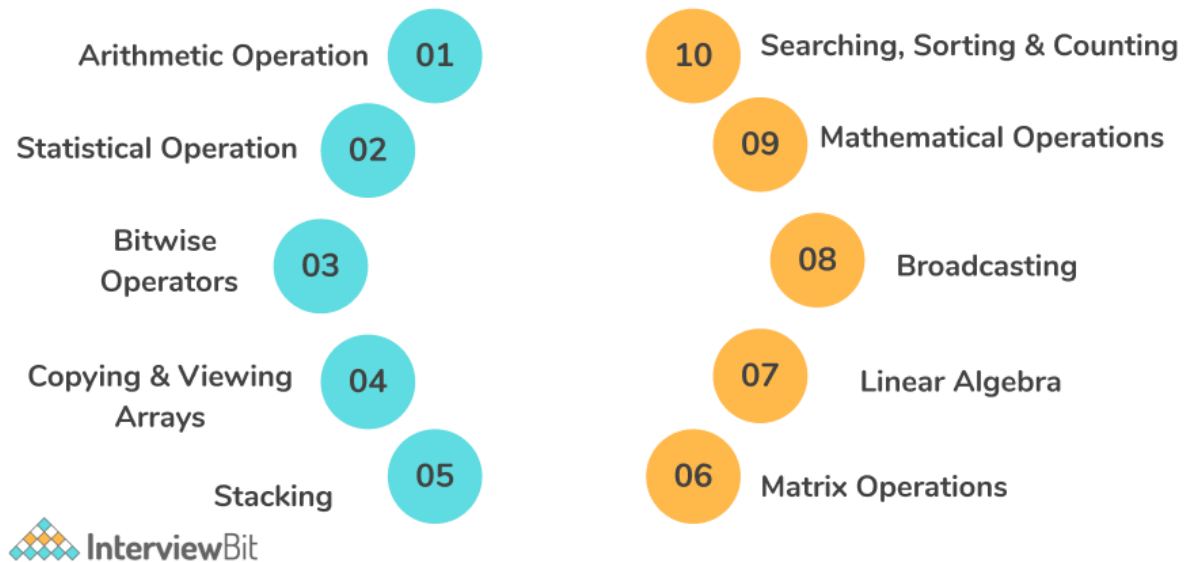
NumPy is widely used in various fields that require fast numerical computations and efficient data processing. Some major applications of NumPy are:

- Data Science: Used for data analysis, preprocessing, and manipulation.
- Machine Learning: Serves as the foundation for libraries such as Scikit-learn, TensorFlow, and PyTorch.
- Artificial Intelligence: Performs numerical computations required for AI algorithms.
- Deep Learning: Handles tensors, matrices, and mathematical operations used in neural networks.
- Scientific Computing: Solves complex mathematical and scientific problems.
- Image Processing: Represents images as arrays and performs operations like filtering and transformation.
- Computer Vision: Used in object detection, image recognition, and video processing.
- Signal Processing: Processes audio signals, digital signals, and communication data.

✓ Features of NumPy :-

- Multidimensional Arrays: NumPy supports powerful N-dimensional arrays using the ndarray object.
- High Performance: NumPy performs numerical operations faster than standard Python lists.
- Memory Efficient: NumPy arrays use less memory for storing large amounts of numerical data.
- Vectorized Operations: Mathematical operations can be performed on entire arrays without writing loops.
- Broadcasting: NumPy can perform operations on arrays of different compatible shapes.
- Mathematical Functions: It provides many built-in mathematical and statistical functions.
- Indexing and Slicing: NumPy provides efficient methods to access and modify array elements.
- Linear Algebra: It supports matrix operations, determinants, inverses, and other linear algebra operations.

Uses Of NumPy



✓ Installing and Importing NumPy

Installing NumPy :-

Before using NumPy, it must be installed in your Python environment.

If you are using **Google Colab**, NumPy is already installed by default, so you do not need to install it.

If you are using a local Python environment, you can install NumPy using the following command:

```
pip install numpy
```

You can also upgrade NumPy to the latest version using:

```
pip install --upgrade numpy
```

Importing NumPy :-

After installing NumPy, it must be imported into your Python program before you can use its functions.

The standard convention is:

```
import numpy as np
```

Here:

- **numpy** is the library name.
- **np** is an alias (short name) used for convenience.
- Using **np** makes the code shorter and easier to read.

For example:

Instead of writing:

```
numpy.array([1, 2, 3])
```

we write:

```
np.array([1, 2, 3])
```

```
# Import NumPy
import numpy as np

# Check the installed version
print("NumPy Version:", np.__version__)

# Print a success message
print("NumPy has been imported successfully!")
```

```
NumPy Version: 2.0.2
NumPy has been imported successfully!
```

✓ Creating NumPy Arrays :-

A NumPy array is a collection of elements stored in a structured format. The main array object in NumPy is called **ndarray (N-dimensional array)**.

NumPy arrays can be one-dimensional, two-dimensional, three-dimensional, or multidimensional.

The `np.array()` function is used to create a NumPy array from a Python list or tuple.

Syntax :-

```
np.array(object)
```

Types of NumPy Arrays Based on Dimensions :-

1. Zero-Dimensional Array (0D)

A zero-dimensional array contains a single value. It is also called a scalar array.

2. One-Dimensional Array (1D)

A one-dimensional array contains elements in a single row or sequence.

Example:

```
[10, 20, 30, 40]
```

3. Two-Dimensional Array (2D)

bold text A two-dimensional array contains rows and columns. It is similar to a matrix.

Example:

```
[[1, 2, 3], [4, 5, 6]]
```

4. Three-Dimensional Array (3D)

A three-dimensional array is an array containing multiple two-dimensional arrays.

5. Multidimensional Array

NumPy supports arrays with more than three dimensions. These are called multidimensional or N-dimensional arrays.

Example :-

```
import numpy as np

# 0D Array
arr0 = np.array(10)
print("0D Array:")
print(arr0)

# 1D Array
arr1 = np.array([10, 20, 30, 40])
print("\n1D Array:")
print(arr1)

# 2D Array
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])
print("\n2D Array:")
print(arr2)

# 3D Array
arr3 = np.array([[[1, 2], [3, 4]],
                 [[5, 6], [7, 8]]])

print("\n3D Array:")
print(arr3)
```

```
0D Array:
10
```

```
1D Array:
```

```
[10 20 30 40]
```

2D Array:

```
[[1 2 3]  
 [4 5 6]]
```

3D Array:

```
[[[1 2]  
   [3 4]]
```

```
[[5 6]  
 [7 8]]]
```

Array Properties in NumPy :-

ndim

Description: ndim returns the number of dimensions of an array.

Example :-

✓ shape

Description: shape returns the dimensions of an array as rows and columns.

Example :-

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr.shape)
```

```
(2, 3)
```

✓ size

Description: size returns the total number of elements in an array.

Example :-

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr.size)
```

```
6
```

✓ dtype

Description: dtype returns the data type of the array elements.

Example :-

```
arr = np.array([1, 2, 3, 4])  
  
print(arr.dtype)  
  
int64
```

Indexing and Slicing in NumPy :-

✓ 1D Indexing

Description: 1D indexing is used to access an element from a one-dimensional array.

Example :-

```
arr = np.array([10, 20, 30, 40])  
  
print(arr[2])  
  
30
```

✓ 2D Indexing

Description: 2D indexing is used to access an element using row and column indices.

Example :-

```
arr = np.array([[1, 2, 3],  
                [4, 5, 6]])  
  
print(arr[1, 2])  
  
6
```

✓ Negative Indexing

Description: Negative indexing accesses elements from the end of the array.

Example :-

```
arr = np.array([10, 20, 30, 40])  
  
print(arr[-1])  
  
40
```

✓ Array Slicing

Description: Slicing is used to access a range of elements from an array.

Example :-

```
arr = np.array([10, 20, 30, 40, 50])  
  
print(arr[1:4])  
  
[20 30 40]
```

✓ Mathematical and Statistical Functions in NumPy :-

NumPy provides many built-in mathematical and statistical functions to perform calculations on arrays efficiently.

These functions are useful for analyzing numerical data and performing mathematical operations.

Mathematical Functions :-

Mathematical functions are used to perform calculations on NumPy arrays.

sum()

The sum() function calculates the total sum of all elements in an array.

min()

The min() function returns the smallest value from an array.

max()

The max() function returns the largest value from an array.

sqrt()

The sqrt() function calculates the square root of each element in an array.

Statistical Functions :-

Statistical functions are used to analyze and summarize numerical data.

mean()

The mean() function calculates the average of all elements in an array.

median()

The `median()` function returns the middle value of the data after arranging the values in sorted order.

`std()`

The `std()` function calculates the standard deviation of the array. Standard deviation shows how much the values are spread from the mean.

`var()`

The `var()` function calculates the variance of the array. Variance measures the spread of data around the mean.

These functions make numerical and statistical calculations easier and faster when working with NumPy arrays.

✓ **Example :-**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("Array:", arr)

print("Sum:", np.sum(arr))
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))
print("Mean:", np.mean(arr))
print("Median:", np.median(arr))
print("Standard Deviation:", np.std(arr))
print("Variance:", np.var(arr))
print("Square Root:", np.sqrt(arr))
```

```
Array: [10 20 30 40 50]
Sum: 150
Minimum: 10
Maximum: 50
Mean: 30.0
Median: 30.0
Standard Deviation: 14.142135623730951
Variance: 200.0
Square Root: [3.16227766 4.47213595 5.47722558 6.32455532 7.07106781]
```

✓ **Joining Arrays in NumPy**

`concatenate()`

Description: `concatenate()` is used to join two or more arrays into a single array.

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

result = np.concatenate((arr1, arr2))
print(result)
```

```
[1 2 3 4 5 6]
```

vstack()

Description: `vstack()` is used to stack arrays vertically (row-wise).

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

result = np.vstack((arr1, arr2))
print(result)
```

```
[[1 2 3]
 [4 5 6]]
```

hstack()

Description: `hstack()` is used to stack arrays horizontally (side by side).

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

result = np.hstack((arr1, arr2))
print(result)
```

```
[1 2 3 4 5 6]
```

✓ **Splitting Arrays in NumPy**

`split()`

Description: `split()` is used to divide an array into multiple equal sub-a

```
arr = np.array([1, 2, 3, 4, 5, 6])

result = np.split(arr, 3)

print(result)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

Start coding or [generate](#) with AI.



Boolean Indexing and Filtering in NumPy

Boolean Indexing

Description: Boolean indexing is used to select elements that satisfy a given condition.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[arr > 25])
```

[30 40 50]

Filtering Even Numbers

Description: Filter all even numbers from a NumPy array.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

print(arr[arr % 2 == 0])
```

[2 4 6]

Multiple Conditions

Description: Apply more than one condition to filter array elements.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[(arr >= 20) & (arr <= 40)])
```

[20 30 40]

Double-click (or enter) to edit



Sorting and Searching in NumPy

sort()

Description: sort() is used to arrange array elements in ascending order.

```
arr = np.array([40, 10, 30, 20])

result = np.sort(arr)

print(result)

[10 20 30 40]
```

where()

Description: where() is used to find the indexes of elements that satisfy a given condition.

```
arr = np.array([10, 20, 30, 40, 50])

result = np.where(arr > 25)

print(result)

(array([2, 3, 4]),)
```

✓ Random Numbers in NumPy :-

rand()

Description: rand() generates random numbers between 0 and 1.

```
print(np.random.rand(3))
```

randint()

Description: randint() generates random integers within a given range.

```
print(np.random.randint(1, 10, 5))
```

choice()

Description: choice() selects a random value from an array.

```
arr = np.array([10, 20, 30, 40])

print(np.random.choice(arr))
```

Examples :-

```
print(np.random.rand(3))
```

```
[0.49784225 0.80148545 0.10585514]
```

```
print(np.random.randint(1, 10, 5))
```

```
[9 6 9 1 2]
```

```
print(np.random.choice(arr))
```

```
10
```

Double-click (or enter) to edit

✓ Copy vs View in NumPy :-

copy()

Description: copy() creates a new independent array, so changes in the original array do not affect the copied array.

```
arr = np.array([1, 2, 3])
```

```
copy_arr = arr.copy()
arr[0] = 10
```

```
print(arr)
print(copy_arr)
```

```
[10  2  3]
[1  2  3]
```

view()

Description: view() creates a new view of the original array, so changes in the original array can affect the view.

```
arr = np.array([1, 2, 3])
```

```
view_arr = arr.view()
arr[0] = 10
```

```
print(arr)
print(view_arr)
```

```
[10  2  3]
[10  2  3]
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr.ndim)
```

```
2
```

Double-click (or enter) to edit

✓ **Broadcasting in NumPy :-**

Description: Broadcasting allows NumPy to perform arithmetic operations on arrays of different but compatible shapes without manually resizing them.

```
import numpy as np

arr = np.array([10, 20, 30])

result = arr + 5

print(result)

[15 25 35]
```

✓ **Broadcasting Between Two Arrays**

Description: Broadcasting automatically expands the smaller array to match the larger array during an operation.

```
import numpy as np

arr1 = np.array([[1], [2], [3]])
arr2 = np.array([10, 20, 30])

result = arr1 + arr2

print(result)

[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

✓ **Matrix Operations in NumPy :-**

Matrix Multiplication (*)

Description: Matrix multiplication combines two matrices to produce a new matrix (use `matmul()` or `dot()` for true matrix multiplication).

```
arr1 = np.array([[1, 2],  
                 [3, 4]])  
  
arr2 = np.array([[5, 6],  
                 [7, 8]])  
  
print(arr1 * arr2)
```

```
[[ 5 12]  
 [21 32]]
```

dot()

Description: dot() performs matrix multiplication between two arrays.

```
arr1 = np.array([[1, 2],  
                 [3, 4]])  
  
arr2 = np.array([[5, 6],  
                 [7, 8]])  
  
print(np.dot(arr1, arr2))
```

```
[[19 22]  
 [43 50]]
```

matmul()

Description: matmul() is used to perform matrix multiplication between two arrays.

```
arr1 = np.array([[1, 2],  
                 [3, 4]])  
  
arr2 = np.array([[5, 6],  
                 [7, 8]])  
  
print(np.matmul(arr1, arr2))
```

```
[[19 22]  
 [43 50]]
```

inverse()

Description: np.linalg.inv() is used to find the inverse of a square matrix.

The inverse exists only for square matrices whose determinant is not equal to 0.

✓ Handling Missing Values in NumPy :-

np.nan

Description: np.nan represents a missing or undefined numerical value.

Example :-

```
arr = np.array([10, 20, np.nan, 40])  
  
print(arr)  
  
[10. 20. nan 40.]
```

isnan()

Description: isnan() checks whether each element is NaN or not.

```
arr = np.array([10, np.nan, 30])  
  
print(np.isnan(arr))  
  
[False  True False]
```

nanmean()

Description: nanmean() calculates the mean while ignoring NaN values.

```
arr = np.array([10, 20, np.nan, 40])  
  
print(np.nanmean(arr))  
  
23.333333333333332
```

nansum()

Description: nansum() calculates the sum while ignoring NaN values.

```
arr = np.array([10, 20, np.nan, 40])  
  
print(np.nansum(arr))  
  
70.0
```

Conclusion :-

NumPy is a powerful Python library that provides fast and efficient tools for working with arrays, numerical calculations, mathematical operations, data analysis, and scientific computing.

NumPy is widely used in Data Science, Machine Learning, Artificial Intelligence, Scientific Computing, and Engineering. Its powerful ndarray object, built-in functions,

broadcasting, indexing, and matrix operations make numerical computations easier and faster.